# Ders 24 Çalışma Özeti

## Ders 24 Çalışma Özeti

### Genel Konular

- Metin içinde alt dizgi arama (substring search): metin (text, "samanlık") ve desen (pattern, "iğne")
- Kaba kuvvet (brute force) arama: mantığı, akış diyagramı, en iyi ve en kötü durum karmaşıklığı
- $n \cdot m$ karmaşıklığında $m$'nin ne zaman sabit sayılabileceği
- Döngü sınırını döngü dışında bir kez hesaplama (döngüden bağımsız işlemleri dışarı alma)
- Boyer-Moore yaklaşımı (hocanın adlandırmasıyla Boyer-Moore-Horspool): sağdan sola karşılaştırma ve atlama (skip)
- Uyuşmayan karakter desende yoksa ve varsa atlama miktarı; negatif atlama durumu
- En sağdaki geçiş (rightmost occurrence) tablosu: `shiftTable`
- Boyer-Moore kodu, örnek üzerinde adım adım izleme ve karmaşıklığı
- Trie (önek ağacı) ve otomatik tamamlama
- Run-length kodlu veride doğrudan değişiklik yapma ve dosya yerine bellekte çalışma

### Hocanın Özellikle Vurguladığı Kısımlar

- Sayı aramada tek bir değer bulunur. Dizgi aramada ise desenin **tamamı** metinde art arda eşleşmelidir.
- Kaba kuvvette eşleşme bozulunca arama, başlanan konumun bir sağından (`i+1`) devam etmek zorundadır. Daha ileri atlanırsa arada başlayan bir eşleşme kaçırılabilir.
- Kaba kuvvette metindeki indis `i` ileri geri oynatılmaz. Desen içinde `j` ilerler, metinde `t[i+j]` ile karşılaştırılır.
- Desen uzunluğu $m$ sabit ve küçükse $n \cdot m$ için $O(n)$ denebilir. Ama $m$'nin nereye kadar büyüyeceği bilinmiyorsa (büyük bir belgede büyük bir metin parçası aramak gibi) karmaşıklık $O(n \cdot m)$ olarak yazılmalıdır.
- Döngü koşulunda her turda değişmeyen bir işlem (`N-M`, `A*B` gibi) varsa onu döngüden önce bir kez hesaplayıp bir değişkene atayın. Bazı derleyiciler bunu kendisi yapar ama her derleyici yapmaz. Fazladan 4 baytlık bir değişkenin bellek maliyeti önemsizdir.
- Algoritmayı tasarlarken önce elle bir örnek üzerinde deneyin. Tasarım bitince gereksiz işlemler için bir tur daha üzerinden geçin.
- Fonksiyondan `return` ile çıkılmıyorsa döngüden `break` ile çıkmayın. Bunun yerine `found` gibi bir bayrak kullanın; okunabilirlik bozulmasın.
- Boyer-Moore'un temel fikri: tek tek ilerlemenin tek sebebi eşleşmeyi kaçırmamaktır. Uyuşmayan karakterin desende nerede geçtiği biliniyorsa eşleşmeyi kaçırmadan daha fazla atlanabilir.
- Uyuşmayan karakterin desende nerede geçtiği her uyuşmazlıkta yeniden aranmamalıdır. Desen program boyunca değişmez. Bu bilgi başta bir kez tabloya yazılır ve sonra tablodan okunur.
- Atlama hiçbir zaman geriye doğru olmamalıdır. Hesaplanan atlama negatifse 1 atlanır.
- Kaba kuvvet günlük kullanımda yeterli olabilir. Ancak en kötü durumun sık beklendiği kritik işlerde (örneğin DNA dizilerinde) daha iyi bir algoritma gerekir.

## Kısa Tekrar Notları

- $n$: metin uzunluğu, $m$: desen uzunluğu.
- Desenin başlayabileceği son konum $n - m$'dir. Döngü `i <= n - m` olana kadar döner.
- Kaba kuvvet:
  - Her `i` için desen soldan sağa karşılaştırılır.
  - En iyi durum: her seferinde ilk karakterde uyuşmazlık olur, yaklaşık $n$ karşılaştırma.
  - En kötü durum: metin `AAAAAAAAAB`, desen `AAAAB` gibi; her seferinde son karakterde bozulur. $O(n \cdot m)$.
- Boyer-Moore (derste anlatılan hâli):
  - Desen metne hizalanır. Karşılaştırma desenin **sonundan başına** doğru yapılır (`j = M-1`'den `j >= 0`'a).
  - Uyuşmayan metin karakteri `c = t[i+j]` olsun.
  - `c` desende yoksa desen `c`'nin bir sağına kaydırılır: atlama `j + 1`.
  - `c` desende varsa desendeki **en sağdaki** `c`, metindeki `c`'nin altına gelecek şekilde kaydırılır: atlama `j - table[c]`.
  - Tabloda desende olmayan karakterler için `-1` tutulur. Böylece tek formül yeterli olur: `skip = j - table[c]`. Karakter yoksa `j - (-1) = j + 1` olur.
  - `skip < 0` ise `skip = 1` yapılır.
- Karmaşıklık:
  - Tipik bir metinde yaklaşık $n/m$ karşılaştırma (her seferinde $m$ karakter atlanabilirse).
  - En kötü durum: metin tamamen `B`, desen `ABBBB` gibi. $O(n \cdot m)$.
- Trie: sözlükteki kelimeler harf harf bir ağaca yerleştirilir. Yazılan önek ağaçta izlenerek tamamlama önerilir.

## Detaylı Açıklamalar

### Dizgi Arama Problemi

- Bir metin içinde bir desen aranır. Örnek: metin `FINDINAHAYSTACKNEEDLEINA`, desen `NEEDLE` (6 karakter).
- Karakterler harf olmak zorunda değildir. Noktalama, rakam ya da herhangi bir karakter olabilir.
- Tek bir sayı aramaktan farkı: desenin tüm karakterlerinin metinde art arda karşılığı bulunmalıdır.

### Kaba Kuvvet Arama

#### Mantık

- Desen metnin başına hizalanır. Karakterler soldan sağa karşılaştırılır.
- Karakterler uyuştukça devam edilir. Desenin sonuna gelinirse desen bulunmuştur.
- Bir karakter uyuşmazsa desen bir sağa kaydırılır ve karşılaştırma desenin başından yeniden başlar.
- Örnek: `ABRA` aranırken `A`, `B` uyuşup üçüncü karakter uyuşmazsa arama bir sonraki konumdan devam eder. Böylece arada başlayan bir `ABRA` kaçırılmaz.

**En Kötü Durum** Metin ve desen tekrarlı karakterlerden oluşursa (ekrandan, 00:18:20):

```
metin : A A A A A A A A A B
desen : A A A A B
        her hizalamada 4 A uyuşur, B'de bozulur;
        desen bir kaydırılır ve aynı şey tekrarlanır
```

Her hizalamada yaklaşık $m$ karşılaştırma yapılır. Hizalama sayısı yaklaşık $n$'dir. Karmaşıklık $O(n \cdot m)$ olur.

**Akış Diyagramı** Tahtadaki akış diyagramı (ekrandan, 00:41:40), adımlar hâlinde:

```
oku: N, M, text[N], pat[M]
NN = N - M
found = 0, i = 0
while (!found && i <= NN)
    j = 0
    while (j < M && pat[j] == text[i+j])
        j++
    if (j == M)  found = 1        // desen bulundu
    else         i++              // bulunamadı, bir sağa kay
if (found)  yaz i
else        yaz "Bulunamadı"
```

- İç döngünün iki çıkış sebebi vardır:
  - `j == M`: desenin bütün karakterleri uyuştu, desen bulundu.
  - `pat[j] != text[i+j]`: eşitlik bozuldu.
- Döngüden çıkınca hangisinin olduğuna `j == M` ile bakılır.
- Örnek: metin `KARGAGARDEDI`, desen `GAK`. `i = 5` (ikinci `G`) iken `G`-`G`, `A`-`A` uyuşur, `R`-`K` bozulur. Arama `i = 6`'dan (`A`) devam eder.

Bir öğrenci, `N-M`'yi bir değişkende tutmanın bellek maliyetini sordu. Hoca şöyle cevapladı: tek bir değişken birkaç bayt yer tutar. Ama döngü milyonlarca kez dönüyorsa her turda çıkarma (daha kötüsü çarpma) yapmak daha pahalıdır. Gömülü sistemlerde de hız genellikle daha kritik olduğundan değeri bir kez hesaplamak daha avantajlıdır.

### Boyer-Moore Yaklaşımı

Boyer ve Moore iki kişidir; algoritmayı onlar ortaya koymuştur. Hoca, Horspool'un (1980 civarı) algoritmayı biraz daha verimli hâle getirdiğini söyledi. Derste anlatılan hâlin adlandırması için "Ders sonrası notlar" bölümüne bakın.

### Fikir

- Karşılaştırma desenin son karakterinden başlar ve sola doğru gider.
- Kaba kuvvette tek tek kaymanın sebebi, arada başlayan bir eşleşmeyi kaçırmamaktır.
- Uyuşmayan metin karakteri desende hiç yoksa, desen o karakteri kapsayan hiçbir hizalamada eşleşemez. Desen o karakterin ötesine atlatılabilir.
- Uyuşmayan karakter desende varsa desen, o karakterin desendeki en sağdaki geçişi metindeki karakterle çakışacak şekilde kaydırılır.

### Durum 1: Uyuşmayan Karakter Desende Yok

- Örnek: `NEEDLE` aranırken `i = 14`, `j = 4`'te uyuşmayan metin karakteri `Y` olsun.
- `Y` desende yoktur. Desen `Y`'nin hemen sağından başlamalıdır.
- Atlama `j + 1 = 5`, yeni `i = 19`.

### Durum 2: Uyuşmayan Karakter Desende Var

- Örnek: `i = 6`, `j = 3`'te uyuşmayan metin karakteri `N` olsun. `NEEDLE` içinde `N` 0. konumdadır.
- Atlama `j - 0 = 3`, yeni `i = 9`.
- Karakter desende birden fazla kez geçiyorsa (`NEEDLE`'da üç `E` var) yalnız **en sağdaki** geçişe bakılır. Böylece eşleşme kaçırılmaz.

### Negatif Atlama

- Örnek: `i = 16`, `j = 3`'te uyuşmayan metin karakteri `E` olsun. `E`'nin desendeki en sağ konumu 5'tir.
- Atlama `3 - 5 = -2` çıkar. Bu, `i`'yi 14'e geri götürür.
- Geriye gitmek anlamsızdır. Bu durumda `i` bir artırılır (`skip = 1`, yeni `i = 17`).

**En Sağdaki Geçiş Tablosu**

- Tablonun boyu alfabe (kod) uzunluğudur. Derste yalnız büyük harfler kullanıldığı için 26 göz yeter. ASCII için 128 göz gerekir.
- Önce tüm gözlere `-1` yazılır. Desende olmayan karakterler `-1` olarak kalır.
- Sonra desen soldan sağa gezilir ve her karakterin gözüne konumu yazılır. Aynı karakter tekrar geçerse eski değerin üzerine yazılır. Böylece en sağdaki geçiş kalır.
- Göz numarası `if` kullanmadan `karakter - offset` ile bulunur. Büyük harfler için `offset = 'A'`; örneğin `'N' - 'A' = 13`, `'B' - 'A' = 1`.

`NEEDLE` için tablo (ekrandan, 02:02:20):

| Karakter | N | E | D | L | Diğerleri |
|---|---|---|---|---|---|
| En sağdaki konum | 0 | 5 | 3 | 4 | -1 |

`E` için tabloya sırayla 1, 2 ve 5 yazılır; en sonda 5 kalır.

Tablo kodu (ekrandan, 02:02:20):

```
void shiftTable(char pattern[], int table[], int offset, int size)
{
    int i, M = strlen(pattern);
    for (i = 0; i < size; i++)
        table[i] = -1;
    for (i = 0; i < M; i++)
        table[pattern[i] - offset] = i;
}
```

**Arama Kodu** Tahtadaki kod (ekrandan, 02:11:45). Ekranda tablo indisi `table[t[i+j]]` yazılıdır; derste büyük harf tablosu kullanıldığı için `offset` çıkarılmış hâli verilmiştir:

```
int BoyerMoore(char p[], char t[])
{
    int i, j, skip = 0, N = strlen(t), M = strlen(p);
    for (i = 0; i <= N - M; i += skip)
    {
        j = M - 1;
        while ((j >= 0) && (p[j] == t[i + j]))
            j--;
        if (j >= 0)                     /* pattern bulunamadı */
        {
            skip = j - table[t[i + j] - 'A'];
            if (skip < 0)
                skip = 1;
        }
        else
            return i;                   /* index in text where p starts */
    }
    return -1;                          /* not found */
}
```

- `j >= 0` ile çıkıldıysa eşitlik bozulmuştur, desen bu hizada yoktur.
- `j == -1` ile çıkıldıysa desenin bütün karakterleri uyuşmuştur. Başlangıç adresi `i` döndürülür.
- Hoca bu fonksiyonda `for` kullandı, çünkü sonuç `return` ile döndürülüyor. `for` satırında ilk değer, artım ve bitiş koşulu bir arada görülür.

Bu kod ve `shiftTable`, aşağıdaki örnekle birlikte `gcc -Wall` ile derlenip çalıştırıldı (ajan testi). Tablo değerleri ve atlamalar tahtadakiyle aynı çıktı; sonuç 15 oldu. Aynı metinde kaba kuvvet de 15 buldu.

**Örnek Üzerinde İzleme** Metin `FINDINAHAYSTACKNEEDLEINA` (indis 0-23), desen `NEEDLE` (ekrandan, 02:16:45):

| `i` | `j` | Karşılaştırma | Uyuşmayan karakter | `skip` | Yeni `i` |
|---|---|---|---|---|---|
| 0 | 5 | `t[5]='N'` ile `E` | `N` (tabloda 0) | $5 - 0 = 5$ | 5 |
| 5 | 5 | `t[10]='S'` ile `E` | `S` (tabloda -1) | $5 - (-1) = 6$ | 11 |
| 11 | 4 | `E`-`E` uyuştu, `t[15]='N'` ile `L` | `N` (tabloda 0) | $4 - 0 = 4$ | 15 |
| 15 | | `E, L, D, E, E, N` hepsi uyuştu | | | `return 15` |

Toplam 10 karakter karşılaştırması yapıldı. Kaba kuvvet aynı işi çok daha fazla karşılaştırmayla yapar.

**Karmaşıklık**

- Tekrarın az olduğu normal bir metinde uyuşmazlık genellikle ilk bir iki karakterde olur. Her seferinde yaklaşık $m$ karakter atlanır. Karşılaştırma sayısı yaklaşık $n/m$'dir.
- En kötü durum: metin tamamen `B`'lerden, desen `ABBBB`'den oluşsun.
  - Her hizalamada `B`'ler uyuşur, en sonda `A` ile `B` uyuşmaz. Yani $m$ karşılaştırma yapılır.
  - Uyuşmayan `B`'nin desendeki en sağ konumu `j`'nin sağında kaldığı için atlama negatif çıkar ve 1'e çekilir.
  - Karmaşıklık $O(n \cdot m)$ olur.
- Bu durum normal metinlerde pek beklenmez. Bu yüzden yöntem kaba kuvvetten daha avantajlıdır.
- Daha gelişmiş dizgi arama yöntemleri 3. sınıftaki Algoritma Analizi dersinde işlenir.

**Trie ve Otomatik Tamamlama**

- Telefonlarda ve mesajlaşma uygulamalarında yazılan kelimenin tamamlanması trie (önek ağacı) ile yapılabilir.
- Sözlükteki kelimeler harf harf ağaca yerleştirilir. Her düğüm bir harftir. Kelimenin son harfinin düğümüne bir değer yazılır.
- Örnek sözlük (ekrandan, 02:28:45): `by`, `sea`, `sells`, `she`, `shells`, `shore`, `the`.
  - Kullanıcı `s`, `h` yazınca ağaçta `s → h` düğümüne gelinir. Altındaki `she`, `shells`, `shore` önerilir.
  - `o` da yazılırsa yalnız `shore` kalır.
  - `b` yazılınca sözlükte bu harfle başlayan tek kelime `by` olduğu için hemen `by` önerilir.

**Run-Length Kodlu Veri ve Bellekte Çalışma**

Bu bölüm derste sorulan bir soru üzerine konuşuldu.

- Run-length kodlu bir görüntüde bir pikseli değiştirmek için tüm görüntüyü açmak (decode etmek) gerekmez.
  - Piksel sırası, satır ve sütun sayısından hesaplanır.
  - Run uzunlukları baştan toplanarak pikselin hangi run içinde olduğu bulunur.
  - Yalnız o run bölünür. Örnek: 3 tane 20'nin ilki 90 olacaksa `3 20` yerine `1 90 2 20` yazılır.
- Dosyanın üzerinde doğrudan değişiklik yapılmaz. Dosya giriş/çıkışı yavaştır.
  - Satır sayısı, sütun sayısı ve veri baştan belleğe (dinamik ayrılan bir diziye) alınır.
  - İşlemler bellekteki dizi üzerinde yapılır.
  - Kullanıcı çıkış seçince sonuç dosyaya geri yazılır.

**Ders sonrası notlar**

- **Kaba kuvvette karşılaştırma sayısı:** Hoca en kötü durum için "(n-m-1) çarpı n" dedi. Doğrusu: $n-m+1$ hizalama vardır ve her birinde en fazla $m$ karşılaştırma yapılır. Toplam $(n-m+1)\cdot m$, yani $O(n\cdot m)$.
- **Derste anlatılan algoritmanın adı:** Derste anlatılan `skip = j - table[c]` kuralı, Boyer-Moore algoritmasının "kötü karakter" (bad character) kuralıdır. Sedgewick ve Wayne'in kitabındaki Boyer-Moore uygulaması tam olarak budur. Horspool'un 1980'deki algoritması ise farklıdır:
  - Atlama miktarı uyuşmayan karaktere değil, her zaman desenin son karakterinin hizasındaki metin karakterine göre belirlenir.
  - Tablo desenin ilk $m-1$ karakterine göre kurulur. Atlama $m-1-(\text{en sağ konum})$, karakter yoksa $m$ olur.
  - Bu yüzden Horspool'da atlama hiçbir zaman negatif çıkmaz.
  - Tam Boyer-Moore algoritması kötü karakter kuralına ek olarak "iyi sonek" (good suffix) kuralını da kullanır.
- **ASCII boyutu:** Hoca önce 255 dedi, bir öğrenci 128 olduğunu düzeltti. Standart ASCII 7 bitlik 128 karakterdir. 8 bitlik genişletilmiş kodlamalarda 256 değer vardır. Tablo karakterle doğrudan indislenecekse (`table[t[i+j]]`), 127'den büyük karakterlerde `char` negatif olabilir. Bu durumda indis `(unsigned char)t[i+j]` olarak alınmalıdır.
- **PGM biçimleri:** Derste PGM'nin hangi kodunun metin, hangisinin ikili olduğu karıştı. Netpbm biçimlerinde:
  - `P2`: metin (ASCII) PGM. Değerler boşluklarla ayrılır.
  - `P5`: ikili (binary) PGM. Değerler bayt bayt dip dibe yazılır.
  - `P4` ise PGM değil, siyah-beyaz PBM'nin ikili biçimidir.
- **Run bölünce komşu run'lar (ajan notu):** Bir pikseli değiştirdikten sonra yeni değer komşu run'ın değeriyle aynıysa iki run birleştirilmelidir. Örnek: `10 90` ve ardından `1 90` oluşursa `11 90` yazılır. Böylece kodlama en kısa hâlinde kalır.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.